



Xuantie Vector Turbo Extension Specification

Version 0.2, 2025-01-17: This document is in development.

Table of Contents

Preamble	1
Copyright and license information	2
Introduction	3
1. Vector Crypto Turbo	4
1.1. vmacc54l.vv	4
1.2. vmacc54l.vs	5
1.3. vmacc54h.vv	6
1.4. vmacc54h.vs	7
2. Vector Arith Turbo	8
2.1. vile.vv	8
2.2. vilo.vv	8
2.3. vgmul.vv	9
2.4. vgmul.vs	11
2.5. vcrcfoldr.vv	12
2.6. vcrcfoldn.vv	14
3. Vector Coder Turbo	17
3.1. vabd.vv	17
3.2. vabd.vx	17
3.3. vabd.vi	18
3.4. vabdu.vv	19
3.5. vabdu.vx	19
3.6. vabdu.vi	20
3.7. vaba.vv	21
3.8. vaba.vx	22
3.9. vaba.vi	22
3.10. vabau.vv	23
3.11. vabau.vx	24
3.12. vabau.vi	25
3.13. vwabd.vv	25
3.14. vwabd.vx	26
3.15. vwabdu.vv	27
3.16. vwabdu.vx	27
3.17. vwaba.vv	28
3.18. vwaba.vx	29
3.19. vwabau.vv	30
3.20. vwabau.vx	31
3.21. vfabd.vv	31

3.22. vfabd.vf	32
encoder list	34
Bibliography	37

Preamble



This document is in the [Development state](#)

Assume everything can change. This draft specification will change before being accepted as standard, so implementations made to this draft specification will likely not conform to the future standard.

Copyright and license information

Copyright © 2024-2025 Alibaba Inc.

Introduction

This document describes the XuanTie vector turbo extension for RISC-V.

This document was generated on 2025-01-17.

Chapter 1. Vector Crypto Turbo

1.1. vmacc54l.vv

Synopsis

Vector Multiply-Add of 54 width unsigned integer - return low half of result

Mnemonic

vmacc54l.vv vd, vs2, vs1, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b111000	vm	vs2	vs1	0b110	vd	0b0001011						

Reserved Encoding

- SEW is any value other than 64

Description

Multiplies unsigned 54-bit integers in each 64bit element of vs2 with the unsigned 54-bit integers in the corresponding 64bit elements of vs1 to form a 108-bit intermediate results. Expand the low 54-bit of the each 108-bit product to a 64-bit unsigned number. The 64-bit unsigned number is added to the 64bit unsigned integer in the corresponding 64bit elements of vd, overwrite addend.

Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Operation

```
for (int i = 0; i < vl; i++) {
    if (VM || vm(i))
        vd[i] = ign_zext64(ign_zext64(vs2[i], 54) * ign_zext64(vs1[i], 54), 54) + vd[i];
}
```

1.2. vmacc54l.vs

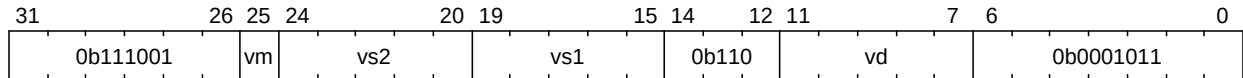
Synopsis

Vector Multiply-Add of 54 width unsigned integer - return low half of result

Mnemonic

vmacc54l.vs vd, vs2, vs1, vm

Encoding



Reserved Encoding

- SEW is any value other than 64
- the vd register groups overlaps the vs1 scalar element group

Description

Multiplies unsigned 54-bit integers in each 64bit element of vs2 with the unsigned 54-bit integers in the corresponding 64bit elements of vs1 to form a 108-bit intermediate results. Expand the low 54-bit of the each 108-bit product to a 64-bit unsigned number. The 64-bit unsigned number is added to the 64bit unsigned integer in the corresponding 64bit elements of vd, overwrite addend.

Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Operation

```
for (int i = 0; i < vl; i++) {
    if (VM || vm(i))
        vd[i] = ign_zext64(ign_zext64(vs2[i], 54) * ign_zext64(vs1[0], 54), 54) + vd[i];
}
```


1.3. vmacc54h.vv

Synopsis

Vector Multiply of 54 width unsigned integer, returning high 54 bits of product and accumulate it

Mnemonic

vmacc54h.vv vd, vs2, vs1, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b111010	vm	vs2	vs1	0b110	vd	0b0001011						

Reserved Encoding

- SEW is any value other than 64

Description

Multiplies unsigned 54-bit integers in each 64bit element of vs2 with the unsigned 54-bit integers in the corresponding 64bit elements of vs1 to form a 108-bit intermediate results. Expand the high 54-bit of the each 108-bit product to a 64-bit unsigned number. The 64-bit unsigned number is added to the 64bit unsigned integer in the corresponding 64bit elements of vd, overwrite addend.

Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Operation

```
for (int i = 0; i < vl; i++) {
    if (VM || vm(i)) {
        uint64_t ZL = (uint128_t)ign_zext64(vs2[i], 54) * ign_zext64(vs1[i], 54) >> 54;
        vd[i] = ign_zext64(ZL, 54) + vd[i];
    }
}
```

1.4. vmacc54h.vs

Synopsis

Vector Multiply of 54 width unsigned integer, returning high 54 bits of product and accumulate it

Mnemonic

vmacc54h.vs vd, vs2, vs1, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b111011	vm	vs2	vs1	0b110	vd	0b0001011						

Reserved Encoding

- SEW is any value other than 64
- the vd register groups overlaps the vs1 scalar element group

Description

Multiplies unsigned 54-bit integers in each 64bit element of vs2 with the unsigned 54-bit integers in the corresponding 64bit elements of vs1 to form a 108-bit intermediate results. Expand the high 54-bit of the each 108-bit product to a 64-bit unsigned number. The 64-bit unsigned number is added to the 64bit unsigned integer in the corresponding 64bit elements of vd, overwrite addend.

Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Operation

```
for (int i = 0; i < vl; i++) {
    if (VM || vm(i)) {
        uint64_t ZL = (uint128_t)ign_zext64(vs2[i], 54) * ign_zext64(vs1[0], 54) >> 54;
        vd[i] = ign_zext64(ZL, 54) + vd[i];
    }
}
```

Chapter 2. Vector Arith Turbo

2.1. vile.vv

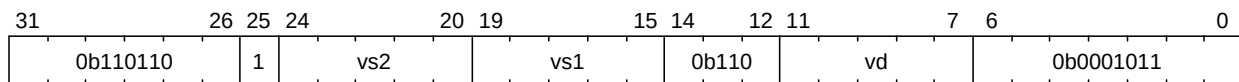
Synopsis

Vector interleave even

Mnemonic

vile.vv vd, vs2, vs1

Encoding



Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

Description

Interleave the even-numbered elements of two vectors for storage to form a new vector data. The even numbered elements of vs1 and vs2 are extracted and the even numbered elements of vs2 are stored in the even numbered positions of vd and the even numbered elements of vs1 are stored in the odd numbered positions of vd. Below is a example schematic diagram of an instruction with lmul=1 and four elements.

src	vs1[3]	vs1[2]	vs1[1]	vs1[0]
src	vs2[3]	vs2[2]	vs2[1]	vs2[0]
dst	vs1[2]	vs2[2]	vs1[0]	vs2[0]

Included in

Xuantie Arithmetic Turbo Extension

2.2. vilo.vv

Synopsis

Vector interleave odd

Mnemonic

vilo.vv vd, vs2, vs1

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b111100	1	vs2	vs1	0b110	vd	0b0001011						

Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding**Description**

Interleave the odd-numbered elements of two vectors for storage to form a new vector data. The odd numbered elements of vs1 and vs2 are extracted and the odd numbered elements of vs2 are stored in the even numbered positions of vd and the odd numbered elements of vs1 are stored in the odd numbered positions of vd. Below is a example schematic diagram of an instruction with lmul=1 and four elements.

src	vs1[3]	vs1[2]	vs1[1]	vs1[0]
src	vs2[3]	vs2[2]	vs2[1]	vs2[0]
dst	vs1[3]	vs2[3]	vs1[1]	vs2[1]

Included in

Xuantie Arithmetic Turbo Extension

2.3. vgmul.vv**Synopsis**

Vector Multiply over GHASH Galois-Field

Mnemonic

vgmul.vv vd, vs2

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b101000	1	vs2	0b10001	0b010	vd	0b1110111						

Vector-Vector Arguments

Register	Direction	Definition
vs2	Input	Source vector 1
vd	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

- SEW is any value other than 8/32



Sew supports 32 as the vector crypto standard, sew for 8 is a new addition of XuanTie Arith Turbo. For sew equal to 32, the command definition is as standard and is not described here. But only the sew8 scenario is described below.



For sew=32, it is necessary to ensure that vl meets the EGW requirement, which is naturally met for sew=8, so there are no mandatory requirements

Description

A GHASH_H multiply is performed. This instruction treats all of the inputs and outputs as 8-bit polynomials and performs operations over GF[2]. It produces the product over GF(2⁸) of the two 8-bit inputs. The multiplication over GF(2⁸) is a carryless multiply of two 8-bit polynomials modulo GHASH's irreducible polynomial ($x^8 + x^4 + x^3 + x^2 + 1$). Since the bits in the bytes are reversed, This instruction internally performs bit swaps within bytes to put the bits in the standard ordering (e.g., 7,6,5,4,3,2,1,0). This instruction must always be implemented such that its execution latency does not depend on the data being operated upon.

Operation

```
function clause execute (VGMUL(vs2, vs1, vd)) = {
  foreach (i from vstart to vl-1) {
    let Y = brev8(get_velem(vd,EGW=8,i)); // Multiplier
    let H = brev8(get_velem(vs2,EGW=8,i)); // Multiplicand
    let Z : bits(8) = 0;
    for (int bit = 0; bit < 8; bit++) {
      if bit_to_bool(Y[bit])
        Z ^= H
    }

    bool reduce = bit_to_bool(H[7]);
    H = H << 1; // left shift H by 1
  }
}
```

```

    if (reduce)
        H ^= 0x1d; // Reduce using x^4 + x^3 + x^2 + 1 polynomial
    }
    let result = brev8(Z);
    set_velem(vd, EGW=8, i, result);
}
RETIRE_SUCCESS
}

```

Included in

Xuantie Arithmetic Turbo Extension

2.4. vgmul.vs**Synopsis**

Vector-Scalar Multiply over GHASH Galois-Field

Mnemonic

vgmul.vs vd, vs2

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b101001	1	vs2	0b10001	0b010	vd	0b1110111						

Vector-Vector(Scalar) Arguments

Register	Direction	Definition
vs2	Input	Source vector 1
vd	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

- SEW is any value other than 8/32
- the vd register group overlaps the vs2 scalar element group



Sew supports 32 as the vector crypto standard, sew for 8 is a new addition of XuanTie Arith Turbo. For sew equal to 32, the command definition is as standard and is not described here. But only the sew8 scenario is described below.



For sew=32, it is necessary to ensure that vl meets the EGW requirement, which is naturally met for sew=8, so there are no mandatory requirements

Description

A GHASH_H multiply is performed. The multipliers are read as vector from vd, the multiplicands subkeys are read from the scalar element group in vs2. The resulting products are written as vector into vd. This instruction treats all of the inputs and outputs as 8-bit polynomials and performs operations over GF[2]. It produces the product over GF(2⁸) of the two 8-bit inputs. The multiplication over GF(2⁸) is a carryless multiply of two 8-bit polynomials modulo GHASH's irreducible polynomial ($x^8 + x^4 + x^3 + x^2 + 1$). Since the bits in the bytes are reversed, This instruction internally performs bit swaps within bytes to put the bits in the standard ordering (e.g., 7,6,5,4,3,2,1,0). This instruction must always be implemented such that its execution latency does not depend on the data being operated upon.

Operation

```
function clause execute (VGMUL(vs2, vs1, vd)) = {
  foreach (i from vstart to vl-1) {
    let Y = brev8(get_velem(vd,EGW=8,i)); // Multiplier
    let H = brev8(get_velem(vs2,EGW=8,0)); // Multiplicand
    let Z : bits(8) = 0;
    for (int bit = 0; bit < 8; bit++) {
      if bit_to_bool(Y[bit])
        Z ^= H

      bool reduce = bit_to_bool(H[7]);
      H = H << 1; // left shift H by 1
      if (reduce)
        H ^= 0x1d; // Reduce using x^4 + x^3 + x^2 + 1 polynomial
    }
    let result = brev8(Z);
    set_velem(vd, EGW=8, i, result);
  }
  RETIRE_SUCCESS
}
```

Included in

Xuantie Arithmetic Turbo Extension

2.5. vcrcfoldr.vv

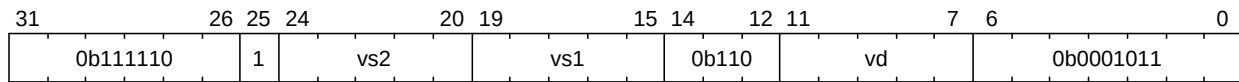
Synopsis

Vector fast fold approach for fast reflection CRC computing

Mnemonic

vcrcfoldr.vv vd, vs2, vs1

Encoding



Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding

- SEW is any value other than 64

Description

Realize the fast input data folding operation for fast reflection CRC computing.

Each SEW-bit element in vd vector register is carry-less multiplied by either each SEW-bit element in vs2 first, generating two intermediate results of interleaved arrangements (if elements in vd / vs2 indexed by [1, 2, 3, 4], the intermediate results shall be [1, 3] and [2, 4]). Inactive elements are replaced with 0 as source operand.

Then perform vector bitwise xor between two intermediate results above and elements in vs1, generating result in vd.



It is recommended that users ensure vl/vstart is an even number, as scenarios with odd numbers do not conform to the original definition of the instruction.

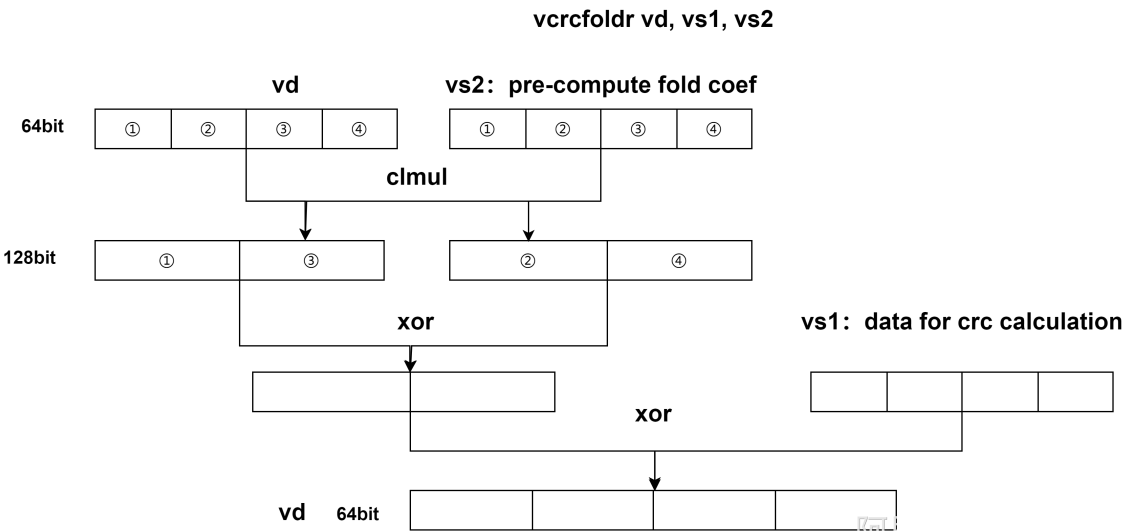
Operation

```
function clause execute (VCRCFOLDR(vs2, vs1, vd, suffix)) = {
  foreach (i from vstart to vl-1) {
    let op1 : bits (SEW) = get_velem(vd,i)
    let op2 : bits (SEW) = get_velem(vs2,i);
    let product1 : bits (SEW) = if i % 2 != 0 then cat(c1mul(op1,op2,SEW),
c1mulh(op1,op2,SEW));
    let product2 : bits (SEW) = if i % 2 == 0 then cat(c1mul(op1,op2,SEW),
c1mulh(op1,op2,SEW));
  }

  foreach (i from vstart to vl-1) {
    let op3 : bits (SEW) = get_velem(vs1,i)
    let result : bits (SEW) = vxor(vxor(product1, product2, SEW), op3, SEW);
    set_velem(vd, i, result);
  }
}
```



```
}  
RETIRE_SUCCESS  
}
```

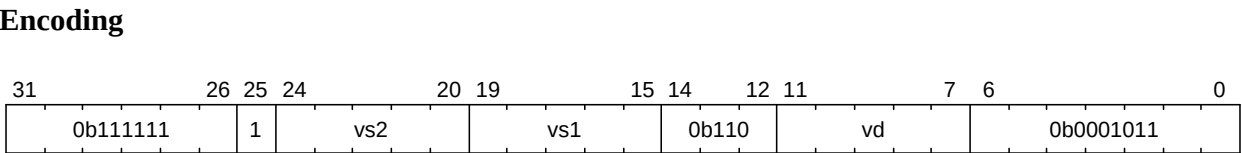


Included in
Xuantie Arithmetic Turbo Extension

2.6. **vcrcfoldn.vv**

Synopsis
Vector fast fold approach for fast norm CRC computing

Mnemonic
vcrcfoldn.vv vd, vs2, vs1



Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding

- SEW is any value other than 64

Description

Realize the fast input data folding operation for fast norm CRC computing.

Each SEW-bit element in vd vector register is carry-less multiplied by either each SEW-bit element in vs2 first, generating two intermediate results of interleaved arrangements (if elements in vd / vs2 indexed by [1, 2, 3, 4], the intermediate results shall be [1, 3] and [2, 4]). Inactive elements are replaced with 0.

Then perform 128 bit vector bitwise xor between two intermediate results above and vector bytes reverse elements in vs1, generating result in vd.

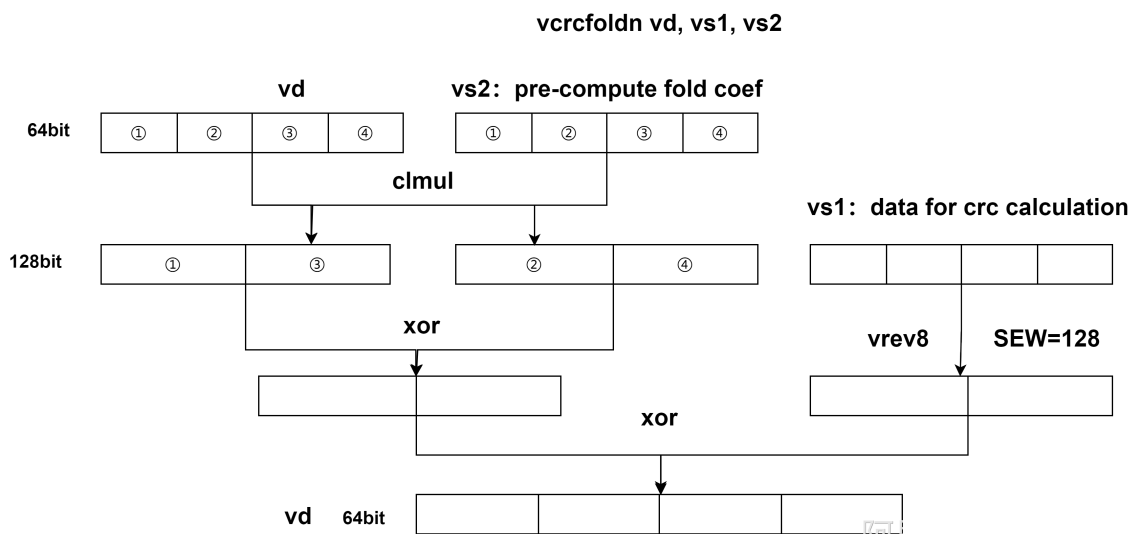


It is recommended that users ensure vl/vstart is an even number, as scenarios with odd numbers do not conform to the original definition of the instruction.

Operation

```
function clause execute (VCRCFOLDN(vs2, vs1, vd, suffix)) = {
  foreach (i from vstart to vl-1) {
    let op1 : bits (SEW) = get_velem(vd,i)
    let op2 : bits (SEW) = get_velem(vs2,i);
    let product1 : bits (SEW) = if i % 2 != 0 then cat(clmul(op1,op2,SEW),
clmulh(op1,op2,SEW));
    let product2 : bits (SEW) = if i % 2 == 0 then cat(clmul(op1,op2,SEW),
clmulh(op1,op2,SEW));
  }

  foreach (i from vstart to vl-1) {
    let op3 : bits (SEW) = get_velem(vs1,i)
    let op3_rev : bits (2 * SEW) = rev8(op3, SEW)
    let result : bits (SEW) = vxor(vxor(product1, product2, SEW), op3_rev,
SEW);
    set_velem(vd, i, result);
  }
  RETIRE_SUCCESS
}
```

**Included in**

Xuantie Arithmetic Turbo Extension

Chapter 3. Vector Coder Turbo

3.1. vabd.vv

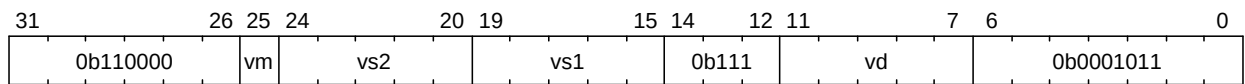
Synopsis

Vector absolute value of the difference

Mnemonic

vabd.vv vd, vs2, vs1, vm

Encoding



Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

Description

Subtract each element of the two source registers and then do an absolute return to the destination registers. Note that the operands are signed numbers. $vd[i] = \text{abs}(vs2[i] - vs1[i])$

Included in

Xuantie Coder Turbo Extension

3.2. vabd.vx

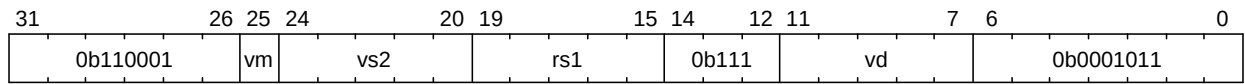
Synopsis

Vector absolute value of the difference

Mnemonic

vabd.vx vd, vs2, rs1, vm

Encoding



Vector-Scalar Arguments

Register	Direction	Definition
rs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

Description

Subtract element of the source vector and the source scalar and then do an absolute return to the destination vector. Note that the operands are signed numbers. $vd[i] = \text{abs}(vs2[i] - rs1)$

Included in

Xuantie Coder Turbo Extension

3.3. vabd.vi

Synopsis

Vector absolute value of the difference

Mnemonic

vabd.vi vd, vs2, imm, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b110010	vm	vs2	imm	0b111	vd	0b0001011						

Vector-Immediate Arguments

Register	Direction	Definition
imm	Input	Source immediate 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

Description

Subtract element of the source vector and the source immediate and then do an absolute return to the destination vector. Note that the operands are signed numbers. $vd[i] = \text{abs}(vs2[i] - \text{imm})$

Included in

Xuantie Coder Turbo Extension

3.4. vabdu.vv

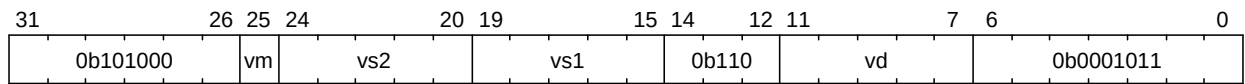
Synopsis

Vector absolute value of the difference

Mnemonic

vabdu.vv vd, vs2, vs1, vm

Encoding



Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

Description

Subtract each element of the two source registers and then do an absolute return to the destination registers. Note that the operands are unsigned numbers. $vd[i] = \text{abs}(vs2[i] - vs1[i])$

Included in

Xuantie Coder Turbo Extension

3.5. vabdu.vx

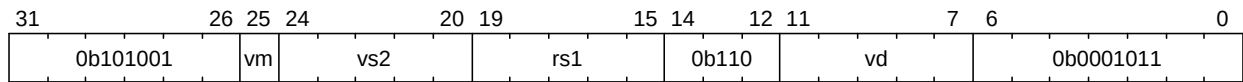
Synopsis

Vector absolute value of the difference

Mnemonic

vabdu.vx vd, vs2, rs1, vm

Encoding



Vector-Scalar Arguments

Register	Direction	Definition
rs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

Description

Subtract element of the source vector and the source scalar and then do an absolute return to the destination vector. Note that the operands are unsigned numbers. $vd[i] = \text{abs}(vs2[i] - rs1)$

Included in

Xuantie Coder Turbo Extension

3.6. vabdu.vi

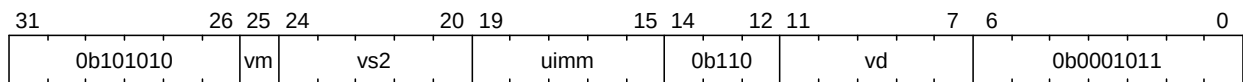
Synopsis

Vector absolute value of the difference

Mnemonic

vabdu.vi vd, vs2, uimm, vm

Encoding



Vector-Immediate Arguments

Register	Direction	Definition
uimm	Input	Source immediate 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

Description

Subtract element of the source vector and the source immediate and then do an absolute return to the destination vector. Note that the operands are unsigned numbers. $vd[i] = \text{abs}(vs2[i] - uimm)$

Included in

Xuantie Coder Turbo Extension

3.7. vaba.vv

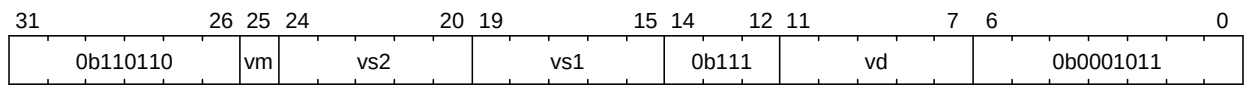
Synopsis

Vector absolute value of the difference and accumulate

Mnemonic

vaba.vv vd, vs2, vs1, vm

Encoding



Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding

Description

Subtract each corresponding element of the two source registers and then do the absolute value, accumulating to the destination registers. Note that the operands are signed numbers. $vd[i] += \text{abs}(vs2[i] - vs1[i])$

Included in

Xuantie Coder Turbo Extension

3.8. vaba.vx

Synopsis

Vector absolute value of the difference and accumulate

Mnemonic

vaba.vx vd, vs2, rs1, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b110111	vm	vs2	rs1	0b111	vd	0b0001011						

Vector-Scalar Arguments

Register	Direction	Definition
rs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding

Description

Subtract each corresponding element of the source vector and the source scalar and then do the absolute value, accumulating to the destination vector. Note that the operands are signed numbers. $vd[i] += \text{abs}(vs2[i] - rs1)$

Included in

Xuantie Coder Turbo Extension

3.9. vaba.vi

Synopsis

Vector absolute value of the difference and accumulate

Mnemonic

vaba.vi vd, vs2, imm, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b110011	vm	vs2	imm	0b111	vd	0b0001011						

Vector-Immediate Arguments

Register	Direction	Definition
imm	Input	Source immediate 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding

Description

Subtract each corresponding element of the source vector and the source immediate and then do the absolute value, accumulating to the destination vector. Note that the operands are signed numbers. $vd[i] += abs(vs2[i] - imm)$

Included in

Xuantie Coder Turbo Extension

3.10. vabau.vv

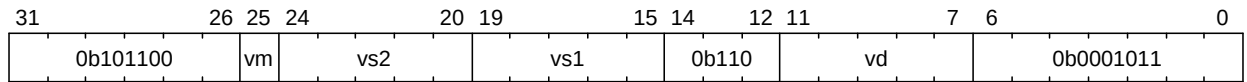
Synopsis

Vector absolute value of the difference and accumulate

Mnemonic

vabau.vv vd, vs2, vs1, vm

Encoding



Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding**Description**

Subtract each corresponding element of the two source registers and then do the absolute value, accumulating to the destination registers. Note that the operands are unsigned numbers.
 $vd[i] += \text{abs}(vs2[i] - vs1[i])$

Included in

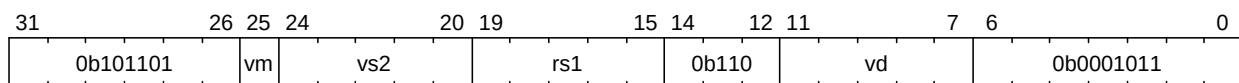
Xuantie Coder Turbo Extension

3.11. vabau.vx**Synopsis**

Vector absolute value of the difference and accumulate

Mnemonic

vabau.vx vd, vs2, rs1, vm

Encoding**Vector-Scalar Arguments**

Register	Direction	Definition
rs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding**Description**

Subtract each corresponding element of the source vector and the source scalar and then do the absolute value, accumulating to the destination vector. Note that the operands are unsigned numbers.
 $vd[i] += \text{abs}(vs2[i] - rs1)$

Included in

Xuantie Coder Turbo Extension

3.12. vabau.vi

Synopsis

Vector absolute value of the difference and accumulate

Mnemonic

vabau.vi vd, vs2, uimm, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b101011	vm	vs2	uimm	0b110	vd	0b0001011						

Vector-Immediate Arguments

Register	Direction	Definition
uimm	Input	Source immediate 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding

Description

Subtract each corresponding element of the source vector and the source immediate and then do the absolute value, accumulating to the destination vector. Note that the operands are unsigned numbers. $vd[i] += \text{abs}(vs2[i] - uimm)$

Included in

Xuantie Coder Turbo Extension

3.13. vwabd.vv

Synopsis

Vector absolute value of the difference obtained by sign-extended signed numbers

Mnemonic

vwabd.vv vd, vs2, vs1, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b111000	vm	vs2	vs1	0b111	vd	0b0001011						

Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

SEW is any value other than 8, 16, 32

Description

After the signed expansion of each element of the two source registers, the corresponding elements are subtracted and then returned to the destination register as absolute values $vd[i] = \text{abs}(\text{signed}(vs2[i]) - \text{signed}(vs1[i]))$

Included in

Xuantie Coder Turbo Extension

3.14. vwabd.vx

Synopsis

Vector absolute value of the difference obtained by sign-extended signed numbers

Mnemonic

vwabd.vx vd, vs2, rs1, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b111001	vm	vs2	rs1	0b111	vd	0b0001011						

Vector-Scalar Arguments

Register	Direction	Definition
rs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

SEW is any value other than 8, 16, 32

Description

After the signed expansion of each element of the two source registers, the corresponding elements are subtracted and then returned to the destination register as absolute values $vd[i] = \text{abs}(\text{signed}(vs2[i]) - \text{signed}(rs1))$

Included in

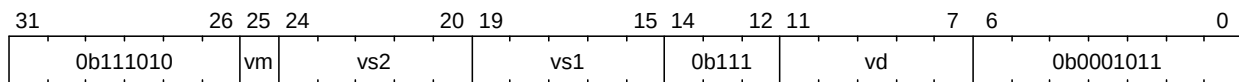
Xuantie Coder Turbo Extension

3.15. vwabdu.vv**Synopsis**

Vector absolute value of the difference obtained by unsign-extended numbers

Mnemonic

vwabdu.vv vd, vs2, vs1, vm

Encoding**Vector-Vector Arguments**

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

SEW is any value other than 8, 16, 32

Description

After the unsigned expansion of each element of the two source registers, the corresponding elements are subtracted and then returned to the destination register as absolute values $vd[i] = \text{abs}(\text{unsigned}(vs2[i]) - \text{unsigned}(vs1[i]))$

Included in

Xuantie Coder Turbo Extension

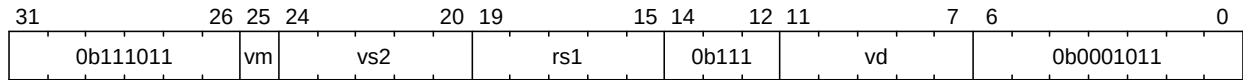
3.16. vwabdu.vx

Synopsis

Vector absolute value of the difference obtained by unsign-extended numbers

Mnemonic

`vwabdu.vx vd, vs2, rs1, vm`

Encoding**Vector-Scalar Arguments**

Register	Direction	Definition
rs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

SEW is any value other than 8, 16, 32

Description

After the unsigned expansion of each element of the two source registers, the corresponding elements are subtracted and then returned to the destination register as absolute values $vd[i] = \text{abs}(\text{unsigned}(vs2[i]) - \text{unsigned}(rs1))$

Included in

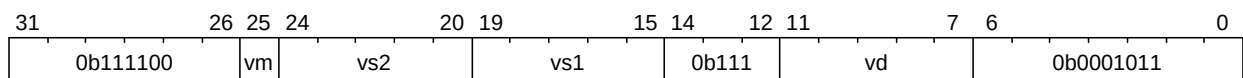
Xuantie Coder Turbo Extension

3.17. vwaba.vv**Synopsis**

Vector absolute value of the difference obtained by sign-extended signed numbers and accumulate

Mnemonic

`vwaba.vv vd, vs2, vs1, vm`

Encoding

Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding

SEW is any value other than 8, 16, 32

Description

After the signed expansion of each element of the two source registers, the corresponding elements are subtracted and then the absolute value is made, and accumulated to the destination registers $vd[i] += \text{abs}(\text{signed}(vs2[i]) - \text{signed}(vs1[i]))$

Included in

Xuantie Coder Turbo Extension

3.18. vwaba.vx

Synopsis

Vector absolute value of the difference obtained by sign-extended signed numbers and accumulate

Mnemonic

vwaba.vx vd, vs2, rs1, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b111101	vm		vs2		rs1	0b111		vd			0b0001011	

Vector-Scalar Arguments

Register	Direction	Definition
rs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding

SEW is any value other than 8, 16, 32

Description

After the signed expansion of corresponding element of the source vector and the source scalar, the corresponding elements are subtracted and then the absolute value is made, and accumulated to the destination registers $vd[i] += \text{abs}(\text{signed}(vs2[i]) - \text{signed}(rs1))$

Included in

Xuantie Coder Turbo Extension

3.19. vwabau.vv**Synopsis**

Vector absolute value of the difference obtained by unsign-extended numbers and accumulate

Mnemonic

vwabau.vv vd, vs2, vs1, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b1111110	vm	vs2	vs1	0b111	vd	0b0001011						

Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding

SEW is any value other than 8, 16, 32

Description

After the unsigned expansion of each element of the two source registers, the corresponding elements are subtracted and then the absolute value is made, and accumulated to the destination registers $vd[i] += \text{abs}(\text{unsigned}(vs2[i]) - \text{unsigned}(vs1[i]))$

Included in

Xuantie Coder Turbo Extension

3.20. vwabau.vx

Synopsis

Vector absolute value of the difference obtained by unsign-extended numbers and accumulate

Mnemonic

vwabau.vx vd, vs2, rs1, vm

Encoding

31	26	25	24	20	19	15	14	12	11	7	6	0
0b111111	vm	vs2	rs1	0b111	vd	0b0001011						

Vector-Scalar Arguments

Register	Direction	Definition
rs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Input	Source vector 3
vd	Output	Destination vector

Reserved Encoding

SEW is any value other than 8, 16, 32

Description

After the unsigned expansion of corresponding element of the source vector and the source scalar, the corresponding elements are subtracted and then the absolute value is made, and accumulated to the destination registers $vd[i] += \text{abs}(\text{unsigned}(vs2[i]) - \text{unsigned}(rs1))$

Included in

Xuantie Coder Turbo Extension

3.21. vfabd.vv

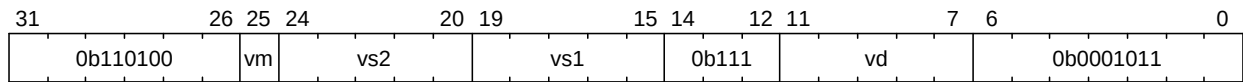
Synopsis

Vector floating-point absolute value of the difference

Mnemonic

vfabd.vv vd, vs2, vs1, vm

Encoding



Vector-Vector Arguments

Register	Direction	Definition
vs1	Input	Source vector 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding

Description

Subtract each floating-point element of the two source registers and then do an absolute return to the destination registers $vd[i] = \text{abs}(vs2[i] - vs1[i])$

Included in

Xuantie Coder Turbo Extension

3.22. vfabd.vf

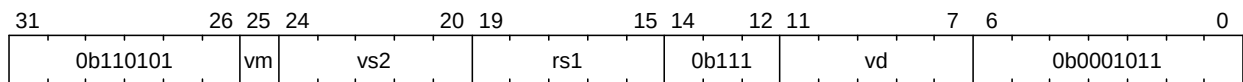
Synopsis

Vector floating-point absolute value of the difference

Mnemonic

vfabd.vf vd, vs2, rs1, vm

Encoding



Vector-Scalar Arguments

Register	Direction	Definition
rs1	Input	Source scalar 1
vs2	Input	Source vector 2
vd	Output	Destination vector

Reserved Encoding**Description**

Subtract each floating-point element of the source vector and the source scalar and then do an absolute return to the destination vector $vd[i] = \text{abs}(vs2[i] - f[rs1])$

Included in

Xuantie Coder Turbo Extension

encoder list

31-26	25	24-20	19-15	14-12	11-7	6-0	inst
xuantie_ai_turbo							
100000	vm	vs2	vs1	110	vd	0001011	vmaqa.vv
100001	vm	vs2	vs1	110	vd	0001011	vmaqa.vx
100010	vm	vs2	vs1	110	vd	0001011	vmaqau.vv
100011	vm	vs2	vs1	110	vd	0001011	vmaqau.vx
100100	vm	vs2	vs1	110	vd	0001011	vmaqasu.vv
100101	vm	vs2	vs1	110	vd	0001011	vmaqasu.vx
100111	vm	vs2	vs1	110	vd	0001011	vmaqaus.vx
100000	vm	vs2	vs1	111	vd	0001011	vpmaqa.vv
100001	vm	vs2	vs1	111	vd	0001011	vpmaqa.vx
100010	vm	vs2	vs1	111	vd	0001011	vpmaqau.vv
100011	vm	vs2	vs1	111	vd	0001011	vpmaqau.vx
100100	vm	vs2	vs1	111	vd	0001011	vpmaqasu.vv
100101	vm	vs2	vs1	111	vd	0001011	vpmaqasu.vx
100111	vm	vs2	vs1	111	vd	0001011	vmaqaus.vx
101000	vm	vs2	vs1	111	vd	0001011	vpnclip.vv
101001	vm	vs2	vs1	111	vd	0001011	vpnclip
101010	vm	vs2	vs1	111	vd	0001011	vpnclipu.vv
101011	vm	vs2	vs1	111	vd	0001011	vpnclipu
101100	vm	vs2	vs1	111	vd	0001011	vpwadd.vv
101101	vm	vs2	vs1	111	vd	0001011	vpwadd
101110	vm	vs2	vs1	111	vd	0001011	vpwaddu.vv
101111	vm	vs2	vs1	111	vd	0001011	vpwaddu
xuantie_crypt_turbo							
111000	vm	vs2	vs1	110	vd	0001011	vmacc54l.vv
111001	vm	vs2	vs1	110	vd	0001011	vmacc54l.vs

31-26	25	24-20	19-15	14-12	11-7	6-0	inst
111010	vm	vs2	vs1	110	vd	0001011	vmacc54h.vv
111011	vm	vs2	vs1	110	vd	0001011	vmacc54h.vs
xuantie_arith_turbo							
101000	1	vs2	1000 1	010	vd	1110111	vgmul.vv
101001	1	vs2	1000 1	010	vd	1110111	vgmul.vs
110110	1	vs2	vs1	110	vd	0001011	vile.vv
111100	1	vs2	vs1	110	vd	0001011	vilo.vv
111110	1	vs2	vs1	110	vd	0001011	vcrcfoldr.vv
111111	1	vs2	vs1	110	vd	0001011	vcrcfoldn.vv
xuantie_coder_turbo							
110000	vm	vs2	vs1	111	vd	0001011	vabd.vv
110001	vm	vs2	vs1	111	vd	0001011	vabd.vx
110010	vm	vs2	imm5	111	vd	0001011	vabd.vi
110100	vm	vs2	vs1	111	vd	0001011	vfabd.vv
110101	vm	vs2	vs1	111	vd	0001011	vfabd.vf
110110	vm	vs2	vs1	111	vd	0001011	vaba.vv
110111	vm	vs2	vs1	111	vd	0001011	vaba.vx
110011	vm	vs2	imm5	111	vd	0001011	vaba.vi
101000	vm	vs2	vs1	110	vd	0001011	vabdu.vv
101001	vm	vs2	vs1	110	vd	0001011	vabdu.vx
101010	vm	vs2	imm5	110	vd	0001011	vabdu.vi
101100	vm	vs2	vs1	110	vd	0001011	vabau.vv
101101	vm	vs2	vs1	110	vd	0001011	vabau.vx
101011	vm	vs2	imm5	110	vd	0001011	vabau.vi
111000	vm	vs2	vs1	111	vd	0001011	vwabd.vv
111001	vm	vs2	vs1	111	vd	0001011	vwabd.vx
111010	vm	vs2	vs1	111	vd	0001011	vwabdu.vv

31-26	25	24-20	19-15	14-12	11-7	6-0	inst
111011	vm	vs2	vs1	111	vd	0001011	vwabdu.vx
111100	vm	vs2	vs1	111	vd	0001011	vwaba.vv
111101	vm	vs2	vs1	111	vd	0001011	vwaba.vx
111110	vm	vs2	vs1	111	vd	0001011	vwabau.vv
111111	vm	vs2	vs1	111	vd	0001011	vwabau.vx

Bibliography